

4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

PROGRAM:

```
import math
import random

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + math.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Training data - XOR
inputs = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]

outputs = [
    [0],
```

```
[1],  
[1],  
[0]  
]
```

```
# Number of neurons
```

```
input_neurons = 2
```

```
hidden_neurons = 4
```

```
output_neurons = 1
```

```
# Learning rate
```

```
learning_rate = 0.5
```

```
# Number of training iterations
```

```
epochs = 10000
```

```
# Initialize weights randomly
```

```
random.seed(1)
```

```
weights_input_hidden = [  
    [random.uniform(-1, 1) for j in range(hidden_neurons)]  
    for i in range(input_neurons)  
]
```

```
weights_hidden_output = [  
    [random.uniform(-1, 1) for j in range(output_neurons)]  
]
```

```

    [random.uniform(-1, 1) for j in range(output_neurons)]
    for i in range(hidden_neurons)
]

# Initialize biases
bias_hidden = [random.uniform(-1, 1) for i in range(hidden_neurons)]
bias_output = [random.uniform(-1, 1) for i in range(output_neurons)]

# Training using Backpropagation
for epoch in range(epochs):

    total_error = 0

    for x, target in zip(inputs, outputs):

        # -----
        # FORWARD PROPAGATION
        # -----

        # Hidden layer
        hidden_input = []

        for j in range(hidden_neurons):
            value = bias_hidden[j]

            for i in range(input_neurons):

```

```

        value += x[i] * weights_input_hidden[i][j]

    hidden_input.append(value)

hidden_output = [
    sigmoid(value) for value in hidden_input
]

# Output layer
output_input = []

for j in range(output_neurons):
    value = bias_output[j]

    for i in range(hidden_neurons):
        value += hidden_output[i] * weights_hidden_output[i][j]

    output_input.append(value)

prediction = [
    sigmoid(value) for value in output_input
]

# -----
# CALCULATE ERROR

```

```
# -----
```

```
error = [  
    target[i] - prediction[i]  
    for i in range(output_neurons)  
]
```

```
total_error += sum(e * e for e in error)
```

```
# -----
```

```
# BACKPROPAGATION
```

```
# -----
```

```
# Output layer delta
```

```
output_delta = [  
    error[i] * sigmoid_derivative(prediction[i])  
    for i in range(output_neurons)  
]
```

```
# Hidden layer delta
```

```
hidden_delta = []
```

```
for i in range(hidden_neurons):
```

```
    error_hidden = 0
```

```
for j in range(output_neurons):  
    error_hidden += (  
        output_delta[j] *  
        weights_hidden_output[i][j]  
    )
```

```
delta = (  
    error_hidden *  
    sigmoid_derivative(hidden_output[i])  
)
```

```
hidden_delta.append(delta)
```

```
# -----
```

```
# UPDATE WEIGHTS
```

```
# -----
```

```
# Hidden -> Output weights
```

```
for i in range(hidden_neurons):  
    for j in range(output_neurons):
```

```
        weights_hidden_output[i][j] += (  
            learning_rate *  
            output_delta[j] *  
            hidden_output[i]
```

)

Output bias

for j in range(output_neurons):

 bias_output[j] += (
 learning_rate * output_delta[j]

)

Input -> Hidden weights

for i in range(input_neurons):

 for j in range(hidden_neurons):

 weights_input_hidden[i][j] += (
 learning_rate *
 hidden_delta[j] *
 x[i]

)

Hidden bias

for j in range(hidden_neurons):

 bias_hidden[j] += (
 learning_rate * hidden_delta[j]

)

```

# Display error every 1000 epochs
if epoch % 1000 == 0:
    print(
        "Epoch:",
        epoch,
        "Error:",
        round(total_error, 6)
    )

# -----
# TESTING THE NETWORK
# -----

print("\nANN Testing Results")
print("-----")

for x in inputs:

    # Hidden layer
    hidden_input = []

    for j in range(hidden_neurons):

        value = bias_hidden[j]

```



```

for i in range(input_neurons):
    value += x[i] * weights_input_hidden[i][j]

hidden_input.append(value)

hidden_output = [
    sigmoid(value) for value in hidden_input
]

# Output layer
output_input = []

for j in range(output_neurons):

    value = bias_output[j]

    for i in range(hidden_neurons):
        value += (
            hidden_output[i] *
            weights_hidden_output[i][j]
        )

    output_input.append(value)

prediction = [

```

```

        sigmoid(value)
    for value in output_input
]

print(
    "Input:",
    x,
    "Predicted Output:",
    round(prediction[0], 4),
    "Class:",
    1 if prediction[0] >= 0.5 else 0
)

```

OUTPUT:

```

>
===== RESTART: E:/Machine Learning/ann_backpropagation EXP4.py =====
Epoch: 0 Error: 1.236329
Epoch: 1000 Error: 0.038121
Epoch: 2000 Error: 0.008191
Epoch: 3000 Error: 0.004312
Epoch: 4000 Error: 0.002875
Epoch: 5000 Error: 0.00214
Epoch: 6000 Error: 0.001696
Epoch: 7000 Error: 0.001401
Epoch: 8000 Error: 0.001191
Epoch: 9000 Error: 0.001035

ANN Testing Results
-----
Input: [0, 0] Predicted Output: 0.0136 Class: 0
Input: [0, 1] Predicted Output: 0.985 Class: 1
Input: [1, 0] Predicted Output: 0.9851 Class: 1
Input: [1, 1] Predicted Output: 0.0168 Class: 0
>

```